UPES, Dehradun

SoCS, Compiler Design, Assignment-III

Note: attempt to answer all questions in legible handwriting.

Q1.

a. State the difference between a compiler and an interpreter.
b. Define the interaction between lexical analyzers and parsers.
c. Enlist the common programming errors which can occur during programming.
d. Enumerate the purpose of LALR parser.
e. Define inherited attribute
f. State the rules of type checking.
g. Illustrate the types of dynamic storage allocations.
h. Explain the role of activation record.
i. Illustrate the causes of redundancy.
j. Explain copy propagation.

Q 2.

a. Divide following program into appropriate lexemes. Which lexemes should get associated lexical values? What should be those values be-

```
float limitedSquare(x) {
/* returns x-squared but never float x; more than 100*/
return (x<= -10.0|| x>=10.0)? 100:x*x;
}
```

b. Describe the roles of lexical, syntax and semantic analyzer.

Q 3.

a. Describe the language denoted by the following regular expressions-
   i. a(a|b)*a
   ii. ((€|a)b*)*
   iii. (a|b)*a(a|b)(a|b)
   iv. a*ba*ba*ba*
   v. !!(aa|bb)*((ab|ba)(aa|bb)*(ab|ba)(aa|bb)*)*
b. Explain the construction of an NFA from a regular expression.

Q 4.

a. Consider the context free grammar

   S -> SS+ | SS* | a

   and the string is aa+a*

   i. Find the leftmost derivation of the string
   ii. Find a rightmost derivation of the string
   iii. Give a parse tree for the string
   iv. Discuss about the ambiguity of the grammar.
b. Explain the error handling and recovery mechanism of syntax analyzer.
c. Show that following grammar is SLR(1) but not LL(1)
   S -> SA | A
   A -> a
d. State the mechanisms to eliminate the left recursion from a grammar.

Q 5.

a. The expressions involving operator + and integer or floating-point operands. Floating point numbers are distinguished by having a decimal point.

E -> E+T | T

T -> num.num | num

Give an SDD to determine the type of each term T and expression E.

b. Explain the S attribute definitions with examples.
c. Translate the arithmetic expression a + (b + c) into
   i. Syntax tree
   ii. Quadruples
   iii. Three address code
d. State and explain the translation of expression and the addressing array elements with examples.

Q 6.

a. What are the issues with the design of code generator? Explain with examples.
b. Explain the stack allocation of space with examples? What principles are helpful when designing the calling sequences.

Q 7.

a. What are induction variables and how to find the induction variables in loops and optimize their computation. Explain with examples.
b. Explain the optimization of basic blocks. How does the directed acyclic graph representation of a basic block facilitate code improving transformations within the represented code.

Q 8.

a. Construct the DAG and identify the value numbers for the subexpressions of the following expressions, assuming + associates from the left.
   i. a + b + (a + b)

ii. a + b + a + b

iii. a + a + (a + a + a + (a + a + a + a))

b. Explain the steps involved in the construction of predictive parser with suitable example.